

Required files (Entities & Enums):

In this section, we extract the core Entities and Enums required for the hotel booking problem statement based on the provided customer enablement and core features.

Entity / Enum Name	Attributes / Values
Role (Enum)	CUSTOMER, ADMIN, RECEPTIONIST
BookingStatus (Enum)	PENDING, CONFIRMED, CANCELLED, CHECKED_IN, CHECKED_OUT
User (Entity)	id, name, email, passwordHash, role
Hotel (Entity)	id, name, location, facilities
Room (Entity)	id, hotelId, roomCategory, price, amenities, isAvailable
Booking (Entity)	id, userId, roomId, reservationNumber, checkInDate, checkOutDate, totalAmount, status
Promotion (Entity)	id, code, discountAmount, type (loyalty, seasonal), expiryDate

Folder Structure

The folder structure is tailored for a scalable full-stack application (Java Spring Boot backend and React + Vite frontend) adhering to the core requirements of maintaining secure APIs and a centralized portal.

Backend:

```
src/main/java/com/hotel/server
├── ServerApplication.java
├── controllers/
│   ├── HotelController.java
│   ├── RoomController.java
│   ├── BookingController.java
│   ├── AuthController.java
│   └── ReceptionController.java
├── models/
│   ├── enums/
│   │   ├── Role.java
│   │   └── BookingStatus.java
│   └── entities/
│       ├── User.java
│       ├── Hotel.java
│       ├── Room.java
│       ├── Booking.java
│       └── Promotion.java
├── repositories/
│   ├── HotelRepository.java
│   ├── RoomRepository.java
│   └── BookingRepository.java
├── services/
│   ├── HotelService.java
│   ├── BookingService.java
│   ├── EmailService.java
│   └── ReceptionService.java
├── security/
│   ├── JwtRequestFilter.java
│   └── RateLimitingFilter.java
├── config/
│   ├── SecurityConfig.java
│   └── SwaggerConfig.java
└── dtos/
    └── SearchRequestDto.java
```

- ↳ BookingResponseDto.java
- ↳ exceptions/
 - ↳ GlobalExceptionHandler.java

Frontend Structure (React + Vite):

- src/
 - ↳ assets/
 - ↳ components/
 - ↳ Navbar.jsx
 - ↳ HotelList.jsx
 - ↳ SearchAndFilter.jsx
 - ↳ BookingForm.jsx
 - ↳ ReceptionistDashboard.jsx
 - ↳ context/
 - ↳ AuthContext.jsx
 - ↳ pages/
 - ↳ Home.jsx
 - ↳ BookingHistory.jsx
 - ↳ CentralizedPortal.jsx
 - ↳ ReceptionPortal.jsx
 - ↳ services/
 - ↳ api.js
 - ↳ bookingService.js
 - ↳ authService.js
 - ↳ receptionService.js
 - ↳ utils/
 - ↳ dateFormatting.js
 - ↳ priceCalculator.js

Business Rules:

The core rules validating operations within the booking system:

- Check-in date must be strictly before the check-out date.

- Room availability status must update instantly upon a confirmed booking.
- Rate limiting must be strictly enforced on all secure APIs (authorization and booking endpoints).
- Only authenticated users can access the quick rebooking option and booking history.
- Promotions (discount codes/loyalty rewards) must be validated for expiry and applicability before recalculating the total.
- Only users with the RECEPTIONIST or ADMIN role can modify booking statuses to CHECKED_IN or CHECKED_OUT.

Actions to be performed according to their Roles:

Role 1: Customer

- Browse hotels, rooms, and amenities with ease.
- Search and filter by location, dates, price range, and amenities.
- Perform a secure and efficient booking process.
- View booking history and initiate quick rebooking.
- Apply seasonal offers and loyalty rewards during checkout.

Role 2: Admin (Centralized Portal Manager)

- Manage centralized portal entities (hotels, room categories, pricing, facilities).
- Oversee confirmed bookings and system-wide availability.
- Generate and monitor active promotional discount codes.
- Assign and manage Receptionist user accounts.

Role 3: Receptionist

- View daily arrivals, departures, and current room occupancy.
- Process customer check-ins and check-outs.
- Modify or cancel existing bookings upon customer request.

- Assign specific physical rooms to confirmed reservations.

Exceptions that should be handled:

- RoomUnavailableException
- InvalidDateRangeException
- RateLimitExceededException
- UnauthorizedAccessException
- InvalidPromoCodeException
- UserNotFoundException
- InvalidBookingStatusTransitionException

Tests need to be done on Endpoints:

- **Endpoint 1 (Search):** Input (Location, Dates, Price Range) → Output (Filtered list of available hotels & rooms)
- **Endpoint 2 (Book):** Input (UserId, RoomId, Dates, AuthToken) → Output (Booking Confirmation Object with Reservation Number)
- **Endpoint 3 (History):** Input (AuthToken) → Output (List of past and active bookings for user)
- **Endpoint 4 (Promotions):** Input (PromoCode) → Output (Discount percentage/amount validation)
- **Endpoint 5 (Check-in/Check-out):** Input (BookingId, NewStatus, ReceptionistAuthToken) → Output (Updated Booking Entity)

Workflow checklist:

- **Workflow-1 (New Booking):** Search & Filter Rooms → Select Room → Authenticate →

Complete Payment/Booking → System Updates Availability → Email Confirmation Triggered.

□ **Workflow-2 (Quick Rebook):** Login → Access Booking History → Select Past Booking → Click Rebook → Choose New Dates → Confirm Booking.

□ **Workflow-3 (Admin Setup):** Admin Login → Access Centralized Portal → Add Hotel Details → Define Room Categories & Pricing → Publish.

□ **Workflow-4 (Front Desk Operations):** Receptionist Login → Access Reception Portal → Verify Customer Reservation Number → Update Status to Checked-In → Assign Room Key.